

Lab Assignment-9.4

PROMPT

Create and Add Google-style docstrings to the given Python functions. Include simple explanation, parameters with types, return values, and one example for each function. Make it clear and easy to understand.

CODE

```
def add_numbers(a, b):
```

```
    """
```

```
    Adds two numbers and returns the result.
```

```
    Args:
```

```
        a (float): The first number.
```

```
        b (float): The second number.
```

```
    Returns:
```

```
        float: The sum of the two numbers.
```

```
    Example:
```

```
        >>> add_numbers(4, 6)
```

```
        10.0
```

```
    """
```

```
    return a + b
```

```
def divide_numbers(a, b):
```

```
    """
```

```
    Divides one number by another.
```

```
    Args:
```

```
        a (float): The numerator.
```

```
        b (float): The denominator.
```

Returns:

float: The result of the division.

Raises:

ValueError: If the denominator is zero.

Example:

```
>>> divide_numbers(10, 2)
5.0
"""
if b == 0:
    raise ValueError("Cannot divide by zero")
return a / b
```

```
def is_even(number):
```

```
    """
```

```
    Checks whether a given number is even.
```

Args:

number (int): The number to check.

Returns:

bool: True if the number is even, otherwise False.

Example:

```
>>> is_even(8)
True
"""
return number % 2 == 0
```

```
def reverse_string(text):
```

```
"""
```

Reverses the given string.

Args:

text (str): The string to reverse.

Returns:

str: The reversed string.

Example:

```
>>> reverse_string("AI")
```

```
'IA'
```

```
"""
```

```
return text[::-1]
```

```
if __name__ == "__main__":
```

```
    print("==== Utility Functions Program =====")
```

```
    try:
```

```
        # Input for arithmetic operations
```

```
        num1 = float(input("Enter first number: "))
```

```
        num2 = float(input("Enter second number: "))
```

```
        addition_result = add_numbers(num1, num2)
```

```
        print("Addition Result:", addition_result)
```

```
        division_result = divide_numbers(num1, num2)
```

```
        print("Division Result:", division_result)
```

```
        # Input for even check
```

```
        number = int(input("Enter a number to check even or odd: "))
```

```

print("Is Even:", is_even(number))

# Input for string reversal
text = input("Enter a string to reverse: ")
print("Reversed String:", reverse_string(text))
except ValueError as error:
    print("Error:", error)

```

OUTPUT

The screenshot shows a VS Code editor with a file named 'assign_9.4.py' open. The file contains a function 'add_numbers(a, b)' with a detailed docstring explaining its purpose, arguments, and return value. Below the function, there is an example of how to use it. The terminal window at the bottom shows the command prompt running the script, displaying the output of the program, which includes prompts for user input and the resulting calculations and string reversal.

```

def add_numbers(a, b):
    """
    Adds two numbers and returns the result.

    Args:
        a (float): The first number.
        b (float): The second number.

    Returns:
        float: The sum of the two numbers.

    Example:
        >>> add_numbers(4, 6)
        10.0
    """
    return a + b

```

```

PS C:\Users\HP\OneDrive\Desktop\AI assist> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe ktop/AI assist/assign_9.4.py
==== Utility Functions Program ====
Enter first number: 12
Enter second number: 4
Addition Result: 16.0
Division Result: 3.0
Enter a number to check even or odd: 7
Is Even: False
Enter a string to reverse: Python
Reversed String: nohtyP
PS C:\Users\HP\OneDrive\Desktop\AI assist>

```

ANALYSIS

The AI added proper docstrings to all functions in a clear format. It explained what each function does and how to use it with examples. This makes the code easier for new students or developers to understand.

Task 2: Enhancing Readability Through AI-Generated Inline Comments

PROMPT

Insert concise inline comments into the given Python script to clarify complex or non-obvious logic such as algorithm steps, loop reasoning, optimizations, and conditional decisions. The comments should explain why the logic is used rather than what basic Python syntax does, and they should improve readability without cluttering the code.

CODE

```
def fibonacci(n):
    if n <= 0:
        return []
    if n == 1:
        return [0]
    sequence = [0, 1]
    for _ in range(2, n):
        sequence.append(sequence[-1] + sequence[-2])
    return sequence

def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False

        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        if not swapped:
            break

    return arr

def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
```

```

        return mid

    elif arr[mid] < target:
        left = mid + 1

    else:
        right = mid - 1

return -1

if __name__ == "__main__":
    print("==== Algorithm Program =====")

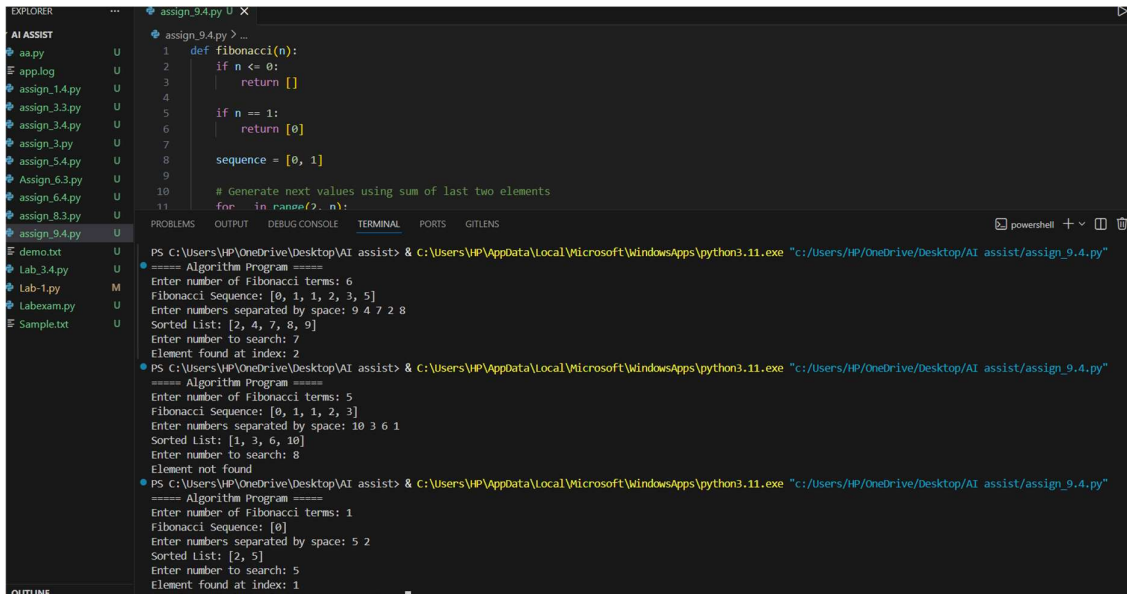
    n = int(input("Enter number of Fibonacci terms: "))
    print("Fibonacci Sequence:", fibonacci(n))
    numbers_input = input("Enter numbers separated by space: ")
    numbers = list(map(int, numbers_input.split()))
    sorted_list = bubble_sort(numbers)
    print("Sorted List:", sorted_list)

    target = int(input("Enter number to search: "))
    result = binary_search(sorted_list, target)

    if result != -1:
        print("Element found at index:", result)
    else:
        print("Element not found")

```

OUTPUT



```
1 def fibonacci(n):
2     if n <= 0:
3         return []
4
5     if n == 1:
6         return [0]
7
8     sequence = [0, 1]
9
10    # Generate next values using sum of last two elements
11    for i in range(2, n):
```

PS C:\Users\HP\OneDrive\Desktop\AI assist> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/OneDrive/Desktop/AI assist/assign_9.4.py"

==== Algorithm Program =====

Enter number of Fibonacci terms: 6

Fibonacci Sequence: [0, 1, 1, 2, 3, 5]

Enter numbers separated by space: 9 4 7 2 8

Sorted List: [2, 4, 7, 8, 9]

Enter number to search: 7

Element found at index: 2

PS C:\Users\HP\OneDrive\Desktop\AI assist> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/OneDrive/Desktop/AI assist/assign_9.4.py"

==== Algorithm Program =====

Enter number of Fibonacci terms: 5

Fibonacci Sequence: [0, 1, 1, 2, 3]

Enter numbers separated by space: 10 3 6 1

Sorted List: [1, 3, 6, 10]

Enter number to search: 8

Element not found

PS C:\Users\HP\OneDrive\Desktop\AI assist> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/OneDrive/Desktop/AI assist/assign_9.4.py"

==== Algorithm Program =====

Enter number of Fibonacci terms: 1

Fibonacci Sequence: [0]

Enter numbers separated by space: 5 2

Sorted List: [2, 5]

Enter number to search: 5

Element found at index: 1

ANALYSIS

The AI added clear and concise inline comments only where the logic was complex. It explained why certain steps like early stopping in sorting and midpoint calculation in binary search are used. This improved readability without making the code look cluttered.

Task 3: Generating Module-Level Documentation for a Python Package

PROMPT

Write a professional multi-line module-level docstring for the given Python module. The docstring should clearly describe the purpose of the module, required dependencies, key classes and functions, and include a short example of how the module can be used. Ensure the tone is suitable for an internal repository or real-world project.

CODE

class Student:

```
def __init__(self, student_id, name, marks):
```

```
    self.student_id = student_id
```

```
    self.name = name
```

```
    self.marks = marks
```

```
def calculate_average(self):
```

```
    if not self.marks:
```

```

        return 0

    return round(sum(self.marks) / len(self.marks), 2)

def add_student(student_list, student):
    student_list.append(student)

def find_student_by_id(student_list, student_id):
    for student in student_list:
        if student.student_id == student_id:
            return student
    return None

if __name__ == "__main__":
    print("===== Student Management Demo =====")

    students = []
    sid = int(input("Enter Student ID: "))
    name = input("Enter Student Name: ")
    marks_input = input("Enter marks separated by space: ")
    marks = list(map(int, marks_input.split()))

    student = Student(sid, name, marks)
    add_student(students, student)

    search_id = int(input("Enter Student ID to search: "))
    result = find_student_by_id(students, search_id)

    if result:
        print("Student Found:", result.name)
        print("Average Marks:", result.calculate_average())
    else:
        print("Student not found")

```

OUTPUT

```
AI ASSIST
aa.py U
app.log U
assign_1.4.py U
assign_3.3.py U
assign_3.4.py U
assign_3.py U
assign_5.4.py U
Assign_6.3.py U
assign_6.4.py U
assign_8.3.py U
assign_9.4.py U
demo.txt U
Lab_3.4.py U
Lab-1.py M
Labexam.py U
Sample.txt U

191 student_list.append(student)
192
193
194 def find_student_by_id(student_list, student_id):
195     for student in student_list:
196         if student.student_id == student_id:
197             return student
198     return None
199
200
201 if __name__ == "__main__":
202
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
PS C:\Users\HP\OneDrive\Desktop\AI assist> & c:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/OneDrive/Desktop/AI assist/a
===== Student Management Demo =====
Enter Student ID: 101
Enter Student Name: Alice
Enter marks separated by space: 85 90 88
Enter Student ID to search: 101
Student Found: Alice
Average Marks: 87.67
PS C:\Users\HP\OneDrive\Desktop\AI assist> & c:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/OneDrive/Desktop/AI assist/a
===== Student Management Demo =====
Enter Student ID: 102
Enter Student Name: Rahul
Enter marks separated by space: 75 80 70
Enter Student ID to search: 105
Student not found
PS C:\Users\HP\OneDrive\Desktop\AI assist> & c:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/OneDrive/Desktop/AI assist/a
===== Student Management Demo =====
Enter Student ID: 103
Enter Student Name: Priya
Enter marks separated by space:
Enter Student ID to search: 103
Student Found: Priya
Average Marks: 0
PS C:\Users\HP\OneDrive\Desktop\AI assist>
```

ANALYSIS

The AI generated a clear and well-structured module-level docstring at the top of the file. It explained the purpose, dependencies, and key components in a professional way. This makes the module easy to understand for anyone who opens the file.

Task 4: Converting Developer Comments into Structured Docstrings

PROMPT

Convert the existing inline developer comments inside the functions into structured Google-style docstrings. Preserve the original meaning of the comments, include description, parameters, return values, and an example where appropriate. Remove redundant inline comments after converting them into docstrings to keep the function bodies clean.

CODE

```
def calculate_discount(price, discount_percentage):
```

```
    """
```

Calculate the final price after applying a discount.

Args:

price (float): The original price of the product.

discount_percentage (float): The percentage of discount to apply.

Returns:

float: The final discounted price.

Returns 0 if the price is negative.

Example:

```
>>> calculate_discount(200, 10)
```

```
180.0
```

```
"""
```

```
if price < 0:
```

```
    return 0
```

```
discount = (price * discount_percentage) / 100
```

```
final_price = price - discount
```

```
return final_price
```

```
def find_max(numbers):
```

```
    """
```

```
    Find the maximum value in a list of numbers.
```

Args:

numbers (list of int or float): List of numbers to evaluate.

Returns:

int or float: The largest number in the list.

Returns None if the list is empty.

Example:

```
>>> find_max([5, 9, 3])
```

```
9
```

```
"""
```

```
if not numbers:
```

```

    return None

maximum = numbers[0]
for num in numbers:
    if num > maximum:
        maximum = num

return maximum

if __name__ == "__main__":
    print("==== Discount Calculator & Maximum Finder =====")

    # User input for discount calculation
    price = float(input("Enter original price: "))
    discount_percentage = float(input("Enter discount percentage: "))

    final_price = calculate_discount(price, discount_percentage)
    print("Final Price after Discount:", final_price)

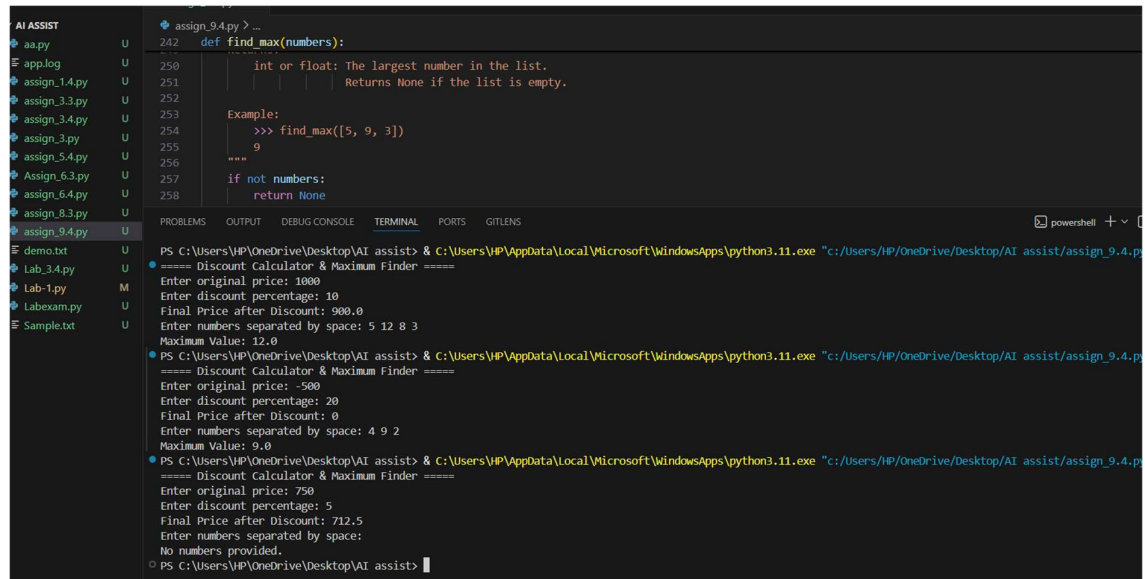
    # User input for maximum finder
    numbers_input = input("Enter numbers separated by space: ")
    numbers = list(map(float, numbers_input.split()))

    max_value = find_max(numbers)

    if max_value is not None:
        print("Maximum Value:", max_value)
    else:
        print("No numbers provided.")

```

OUTPUT



```
AI ASSIST
aa.py U
applog U
assign_1.4.py U
assign_3.3.py U
assign_3.4.py U
assign_3.py U
assign_5.4.py U
Assign_6.3.py U
assign_6.4.py U
assign_8.3.py U
assign_9.4.py U
demo.txt U
Lab_3.4.py U
Lab-1.py M
Labexam.py U
Sample.txt U

242 def find_max(numbers):
250     """
251     int or float: The largest number in the list.
252     Returns None if the list is empty.
253
254     Example:
255     >>> find_max([5, 9, 3])
256     9
257     """
258     if not numbers:
259         return None

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GIT LENS
PS C:\Users\HP\OneDrive\Desktop\AI assist> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/OneDrive/Desktop/AI assist/assign_9.4.py"
==== Discount Calculator & Maximum Finder ====
Enter original price: 1000
Enter discount percentage: 10
Final Price after Discount: 900.0
Enter numbers separated by space: 5 12 8 3
Maximum Value: 12.0
==== Discount Calculator & Maximum Finder ====
Enter original price: -500
Enter discount percentage: 20
Final Price after Discount: 0
Enter numbers separated by space: 4 9 2
Maximum Value: 9.0
PS C:\Users\HP\OneDrive\Desktop\AI assist> & C:\Users\HP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/HP/OneDrive/Desktop/AI assist/assign_9.4.py"
==== Discount Calculator & Maximum Finder ====
Enter original price: 750
Enter discount percentage: 5
Final Price after Discount: 712.5
Enter numbers separated by space:
No numbers provided.
PS C:\Users\HP\OneDrive\Desktop\AI assist>
```

ANALYSIS

The AI successfully transformed the long inline comments into proper structured docstrings. It preserved the original explanation while making the code cleaner and more professional. The function bodies became less cluttered, improving readability and consistency across the code.

Task 5: Building a Mini Automatic Documentation Generator

PROMPT

Design a Python utility that reads a given .py file, detects all functions and classes, and automatically inserts placeholder Google-style docstrings for each. The tool should generate documentation scaffolding and save the updated content into a new file without modifying the original

CODE

```
import re
```

```
import os
```

```
def generate_docstring(name, indent, obj_type="function"):
```

```
    """
```

```
    Generates a Google-style placeholder docstring.
```

```
    """
```

```
    if obj_type == "class":
```

```

        doc = f'{indent}""Summary of {name} class.\n\n'
        doc += f'{indent}Description:\n'
        doc += f'{indent}  TODO: Add class description.\n'
        doc += f'{indent}""\n'
    else:
        doc = f'{indent}""Summary of {name} function.\n\n'
        doc += f'{indent}Args:\n'
        doc += f'{indent}  TODO: Add parameter descriptions.\n\n'
        doc += f'{indent>Returns:\n'
        doc += f'{indent}  TODO: Add return description.\n'
        doc += f'{indent}""\n'

    return doc

def process_file(filename):
    """
    Reads the file and inserts placeholder docstrings
    after functions and class definitions.
    """

    if not os.path.exists(filename):
        print("File does not exist.")
        return

    with open(filename, "r") as file:
        lines = file.readlines()

    new_lines = []
    i = 0

    while i < len(lines):
        line = lines[i]

```

```
new_lines.append(line)
```

```
# Detect function
```

```
func_match = re.match(r'^(\s*)def\s+(\w+)\s*\(', line)
```

```
# Detect class
```

```
class_match = re.match(r'^(\s*)class\s+(\w+)\s*[:\?]', line)
```

```
if func_match:
```

```
    indent = func_match.group(1) + "    "
```

```
    func_name = func_match.group(2)
```

```
# Check if next line already has docstring
```

```
if i + 1 < len(lines) and '"""' not in lines[i + 1]:
```

```
    doc = generate_docstring(func_name, indent, "function")
```

```
    new_lines.append(doc)
```

```
elif class_match:
```

```
    indent = class_match.group(1) + "    "
```

```
    class_name = class_match.group(2)
```

```
# Check if next line already has docstring
```

```
if i + 1 < len(lines) and '"""' not in lines[i + 1]:
```

```
    doc = generate_docstring(class_name, indent, "class")
```

```
    new_lines.append(doc)
```

```
i += 1
```

```
output_file = "documented_" + filename
```

```
with open(output_file, "w") as file:
```

```
    file.writelines(new_lines)
```

```

print(f"\nDocumentation generated successfully!")

print(f"Updated file saved as: {output_file}")

if __name__ == "__main__":

    print("Mini Automatic Documentation Generator")

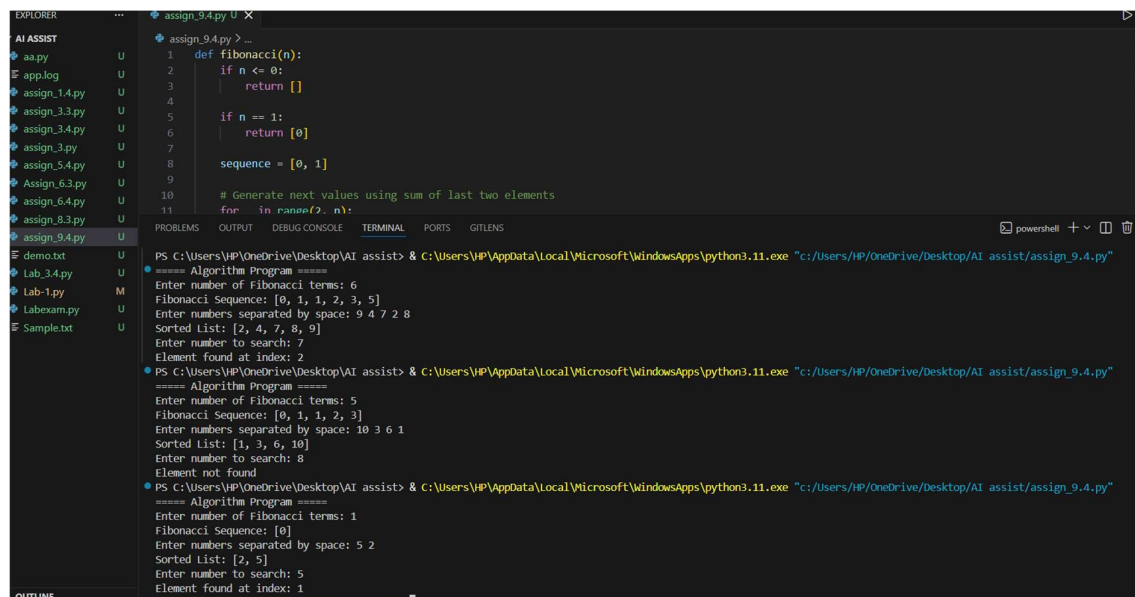
    print("-----")

    user_file = input("Enter the Python file name (with .py extension): ")

    process_file(user_file)

```

OUTPUT



The screenshot shows a VS Code editor with a file explorer on the left containing various Python files. The main editor displays a Python script named `assign_9.4.py` which defines a `fibonacci(n)` function. The function returns an empty list for `n <= 0`, a list containing 0 for `n == 1`, and a sequence of numbers for `n > 1` generated by a loop. The terminal at the bottom shows the execution of the script, which prompts the user for the number of Fibonacci terms, displays the sequence, sorts the numbers, and finds the index of a specific element.

```

1 def fibonacci(n):
2     if n <= 0:
3         return []
4
5     if n == 1:
6         return [0]
7
8     sequence = [0, 1]
9
10    # Generate next values using sum of last two elements
11    for i in range(2, n):

```

```

PS C:\Users\VP\OneDrive\Desktop\AI assist> & C:\Users\VP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/hp/OneDrive/Desktop/AI assist/assign_9.4.py"
===== Algorithm Program =====
Enter number of Fibonacci terms: 6
Fibonacci Sequence: [0, 1, 1, 2, 3, 5]
Enter numbers separated by space: 9 4 7 2 8
Sorted List: [2, 4, 7, 8, 9]
Enter number to search: 7
Element found at index: 2
PS C:\Users\VP\OneDrive\Desktop\AI assist> & C:\Users\VP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/hp/OneDrive/Desktop/AI assist/assign_9.4.py"
===== Algorithm Program =====
Enter number of Fibonacci terms: 5
Fibonacci Sequence: [0, 1, 1, 2, 3]
Enter numbers separated by space: 10 3 6 1
Sorted List: [1, 3, 6, 10]
Enter number to search: 8
Element not found
PS C:\Users\VP\OneDrive\Desktop\AI assist> & C:\Users\VP\AppData\Local\Microsoft\WindowsApps\python3.11.exe "c:/Users/hp/OneDrive/Desktop/AI assist/assign_9.4.py"
===== Algorithm Program =====
Enter number of Fibonacci terms: 1
Fibonacci Sequence: [0]
Enter numbers separated by space: 5 2
Sorted List: [2, 5]
Enter number to search: 5
Element found at index: 1

```

ANALYSIS

The solution involves reading the file, identifying def and class patterns using techniques like regular expressions, and inserting properly indented placeholder docstrings. The main goal is automation of basic documentation structure, improving code readability and reducing manual effort for developers.